

Tas: files de priorité et tri par tas

BUT Informatique - S5 - Qualité algorithmique

1 Construction d'un tas

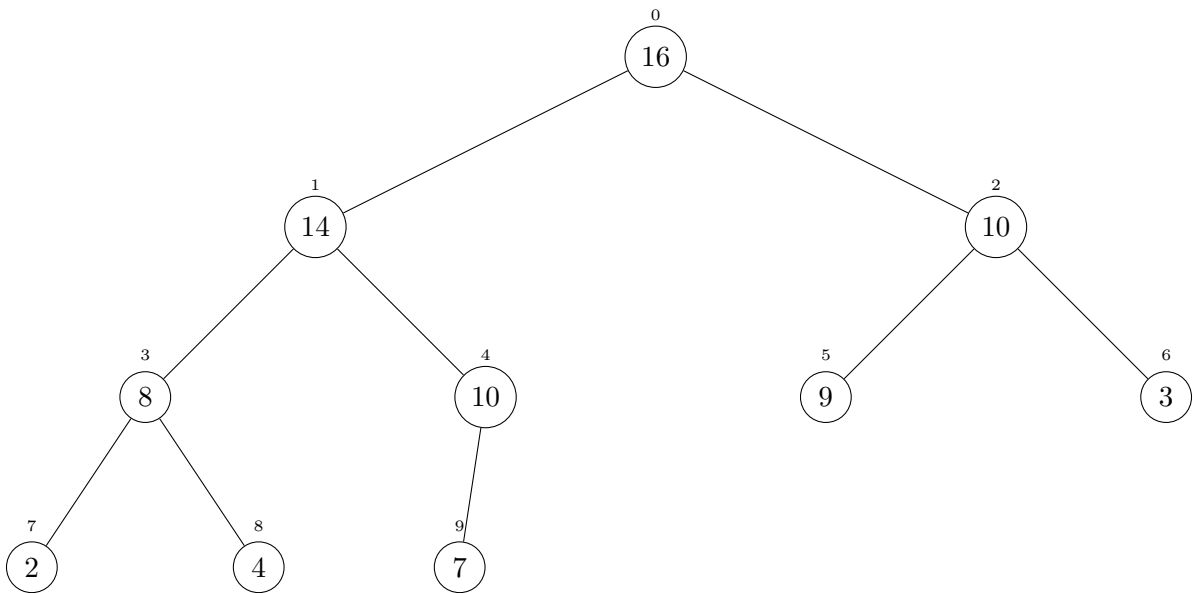


Figure 1: Exemple d'arbre représentant un tas

Un tas est une structure de donnée arborescente contenant un ensemble d'éléments ordonné. Ils sont généralement utilisés pour gérer des files de priorité, dans des algorithmes de parcours de graphes mais aussi pour réaliser des tris (*tri par tas*). Un tas permet d'accéder au plus grand élément en temps constant ($O(1)$).

Plus précisément, un tas est un arbre binaire qui satisfait deux propriétés:

- tout élément, à l'exception de la racine, est inférieur ou égal que son noeud parent;
- l'arbre est presque complet, c'est-à-dire tous les niveaux sont remplis sauf, éventuellement, le dernier partiellement rempli de gauche à droite (voir figure 1).

Ainsi, si l'on divise l'ensemble des noeuds en lignes suivant leur hauteur, on obtient une ligne 0 composée simplement de la racine, puis une ligne 1 contenant au plus deux noeuds, et ainsi de suite.

Question 1

Combien de noeuds contient la ligne i ?

2^i **Question 2**

Dans un arbre presque complet les lignes, exceptée peut-être la dernière, contiennent toutes un nombre maximal de noeuds. De plus les feuilles de la dernière ligne se trouvent toutes à gauche, ainsi tous les noeuds internes sont binaires, excepté le plus à droite de l'avant dernière ligne qui peut ne pas avoir de fils droit. Les feuilles sont toutes sur la dernière et éventuellement l'avant dernière ligne.

On peut numéroté cet arbre en largeur d'abord, c'est-à-dire dans l'ordre donné par les niveaux de l'arbre. Dans la figure 1, les index sont indiqués au-dessus de chaque noeud. Cette numérotation permet d'implémenter les tas à l'aide de tableaux où le numéro de chaque noeud donne l'indice de l'élément du tableau contenant sa valeur. Ainsi, le tas de la figure 1 peut être implanté par le tableau a suivant:

i	0	1	2	3	4	5	6	7	8	9
$a[i]$	16	14	10	8	10	9	3	2	4	7

Pour chacun des arbres suivants, dites si c'est un tas. Si oui, donnez sa représentation en tableau, sinon expliquer pourquoi il ne respecte pas la structure de tas.

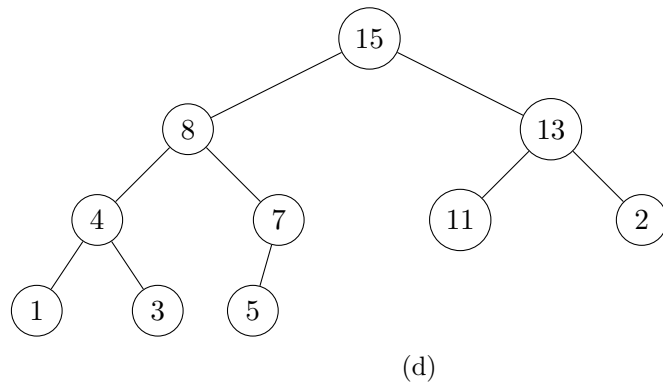
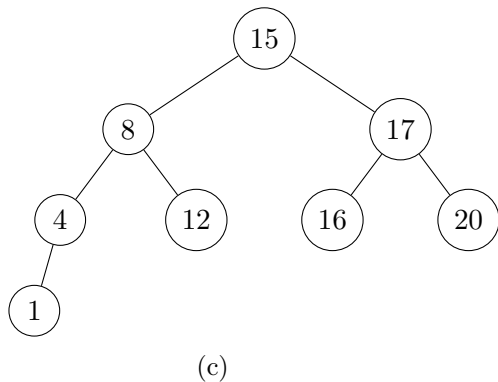
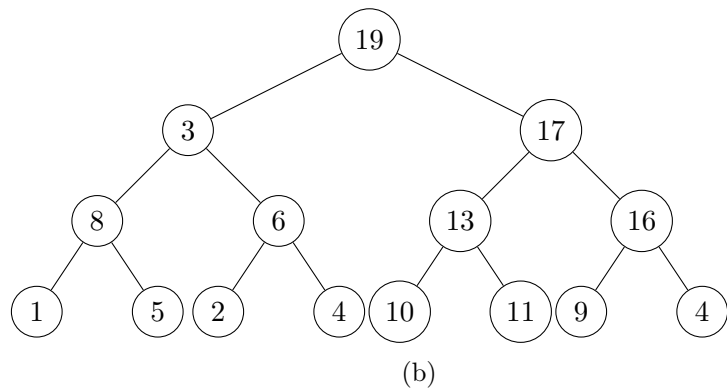
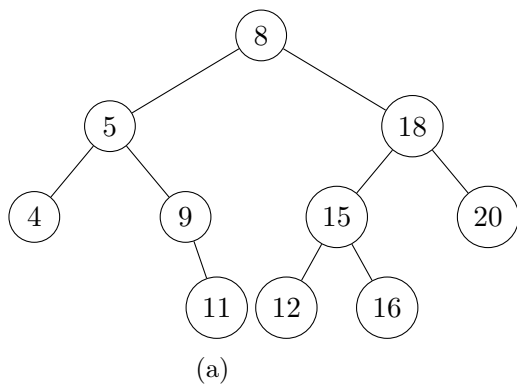
Dans la suite, pour pouvoir ajouter des éléments à un tas, on supposera que le tableau a peut être plus grand le nombre d'éléments dans le tas. On distinguera ainsi deux attributs: $a.\text{longueur}$ qui est la longueur du tableau et $a.\text{taille}$ qui est le nombre d'éléments dans le tas, et tels que $a.\text{taille} \leq a.\text{longueur}$.

SOLUTION

- a) ce n'est pas un tas: pas complet.
- b) ce n'est pas un tas: 3 a un fils plus grand (8)
- c) ce n'est pas un tas: 8 a un fils plus grand (12)
- d) c'est un tas: $\{15, 8, 13, 4, 7, 11, 2, 1, 3, 5\}$

Question 3

L'implantation d'un tas avec un tableau nécessite de trouver la relation entre l'indice i d'un noeud dans le tableau et les indices de ses parent, fils gauche et droit. Donner les algorithmes `parent(i)`, `gauche(i)` et `droite(i)` qui à l'indice i d'un noeud du tableau renvoie l'indice de, respectivement, son parent, fils gauche et fils droit.



SOLUTION

```
gauche(i) = 2*i+1
droite(i) = 2*i+2
parent(i) = floor((i-1)/2)
```

On peut par exemple trouver ces relations empiriquement et prouver que c'est correct par induction sur i . Par exemple, pour montrer que $gauche(i) = 2i + 1$:

- $i = 0$: c'est vrai car $gauche(i) = 1$.
- (HI) $i = k$: on suppose que $gauche(i) = 2i + 1$ pour tout $0 \leq i \leq k$ pour un certain k .
- $i = k + 1$: le fils gauche de $k + 1$ est nécessairement à 2 positions plus loin que le fils gauche de k , donc $gauche(k + 1) = gauche(k) + 2$. Or, par HI, $gauche(k) = 2k + 1$, donc $gauche(k) + 2 = 2k + 1 + 2 = 2(k + 1) + 1$ ce qui montre que $gauche(k + 1) = 2(k + 1) + 1$.

Question 4

Soit a un tableau dont l'élément d'indice i ne respecte pas les contraintes de tas, $entasser_max(a, i)$ déplace cet élément pour obtenir un tas. Plus précisément, si les

deux sous-arbres enracinés en i sont des tas mais que l'élément d'indice i est plus petit qu'un de ses deux fils, alors il est récursivement descendu dans l'arbre jusqu'à obtenir un tas. Expérimentez ce principe sur l'arbre (c) puis sur l'arbre (b) de la question 2. Écrivez cet algorithme.

SOLUTION

```
entasser_max(a,i) {
    l = gauche(i)
    r = droite(i)
    // on détermine dans max l'élément maximum parmi i et ses 2 fils directs
    if (l < a.taille and a[l] > a[i])
        max = l
    else max = i
    if (r < a.taille and a[r] > a[max])
        max = r
    if max != i
        échanger a[i] et a[max]
        entasser_max(a,max)
}
```

Question 5

Quel est, intuitivement, le coût de cet algorithme dans le pire cas ?

SOLUTION

Pour un tableau de taille n , au pire, l'algorithme parcourt toute la hauteur de l'arbre, soit $O(\log n)$.

Question 6

Par construction des tas sous forme de tableau, on remarque que les éléments d'indices $\lceil n/2 \rceil$ à $n-1$ (où n est la taille du tableau) sont tous des feuilles de l'arbre, c'est-à-dire des tas à un élément. Pour convertir un tableau quelconque en un tableau qui vérifie les contraintes de tas, écrivez un algorithme `construire_tas_max(a)` qui parcourt les noeuds internes (c'est-à-dire non feuilles) et appelle `entasser_max` sur chacun de ces noeuds. Commencez par expérimenter cette idée sur l'arbre binaire implémenté par le tableau suivant:

i	0	1	2	3	4	5	6	7	8	9
$a[i]$	4	1	3	2	16	9	10	14	8	7

SOLUTION

```
construire_tas_max(a) =
    a.taille = a.longueur
    for i = ceil(a.longueur/2)-1 downto 1
        entasser_max(a,i)
```

Question 7

Donner un majorant du coût de cet algorithme.

SOLUTION

Chaque appel de `entasser_tas` coûte $O(\log(n))$ et il y a $O(n)$ appels, donc le coût total est $O(n \log n)$. Avec un calcul plus précis on peut montrer que cet algorithme est en fait $O(n)$.

2 Tri par tas

Question 8

Écrire un algorithme `tri_par_tas(a)` qui trie le tableau `a` à l'aide d'une structure de tas.

SOLUTION

```
tri_par_tas(a) =  
    construire_tas_max(a)  
    for i = a.longueur-1 downto 1  
        échanger a[0] et a[i]  
        a.taille = a.taille-1  
        entasser_max(a,1)
```

Question 9

Quel est le coût de cet algorithme ?

SOLUTION

L'appel à `entasser_max` a un coût $O(n)$ et chacun des $n-1$ appels à `entasser_max` prend un temps $O(\log n)$ donc, `tri_par_tas` a un coût $O(n \log n)$.

3 Files de priorité

Question 10

Écrivez l'algorithme `maximum_tas(a)` qui renvoie la valeur du plus grand élément du tableau `a` organisé en tas.

SOLUTION

```
// trivial...
maximum_tas(a)=
    return a[0]
```

Question 11

Quel est le coût de votre algorithme ?

SOLUTION

$O(1)$

Question 12

Écrivez l'algorithme `extraire_max_tas(a)` qui renvoie le plus grand élément du tableau `a` organisé en tas et l'enlève.

SOLUTION

```
// Pour supprimer le plus grand élément, il suffit de le remplacer
// par le dernier et d'appeler entasser_max(a,0) pour le replacer à la bonne position
extraire_max_tas(a) =
    if a.taille < 1
        erreur
    max = a[0]
    a[0] = a[a.taille-1]
    a.taille = a.taille - 1
    entasser_max(a,0)
    return max
```

Question 13

Quel est le coût de votre algorithme ?

SOLUTION

$O(\log n)$

Question 14

Écrire l'algorithme `augmenter_cle(a,i,k)` qui augmente la valeur du noeud `i` à la valeur `k` (si le noeud `i` avait déjà une valeur au moins égale à `k` alors une erreur est produite).

SOLUTION

```
// Il suffit de remonter le noeud i tant que son père a une valeur plus petite que i.
augmenter_cle(a,i,k) =
  if k < a[i]
    erreur
  a[i] = k
  while (i > 0 and a[parent(i)] < a[i])
    permuter a[i] et a[parent(i)]
    i = parent(i)
```